

Psychometric Platform — UML Architecture & REST Endpoint Reference

An Exhaustive Technical Specification of Backend Entities, Services, Security Controllers, and Request Lifecycles

Package: com.psychometric.platform

Framework: Spring Boot 3.4+ / Java 21+

Audience: Backend Engineers, Architects & API Consumers

TABLE OF CONTENTS

- Codebase Inventory & Module Overview
- UML Class Architecture & Domain Diagrams
- Authentication & OTP API Reference
- Admin Management API Reference
- Candidate Management API Reference
- Architectural Observations & Code Quality Findings

1 Module Overview & Codebase Inventory

The backend is structured into modular feature packages under `com.psychometric.platform`, separating core security, identity, administrative management, and caching infrastructure.

1. Authentication & Identity Module (`features.auth`)

Orchestrates candidate and administrator authentication via stateless HMAC-SHA256 JWTs. Implements Redis-backed 6-digit OTP verification for self-service registration and password recovery with 5-minute TTL expirations.

2. User & Candidate Management Module (`features.user`)

Manages the `User` entity lifecycle with dual administration layers: strict `ROLE_SUPER_ADMIN` governance for administrator provisioning, and `ROLE_ADMIN` access for candidate directory management and soft-deactivation.

3. Common Security & Exception Layer (`common.*`)

Provides Spring Security configuration, Bearer token extraction filters, BCrypt hashing, XSS sanitization (`HtmlSanitizer`), and centralized error handling via `GlobalExceptionHandler`.

4. Infrastructure Layer (`infrastructure.*`)

Houses in-memory Redis caching (`RedisTemplate<String, String>`) and an asynchronous email notification engine (`EmailService`) powered by Thymeleaf HTML templates.

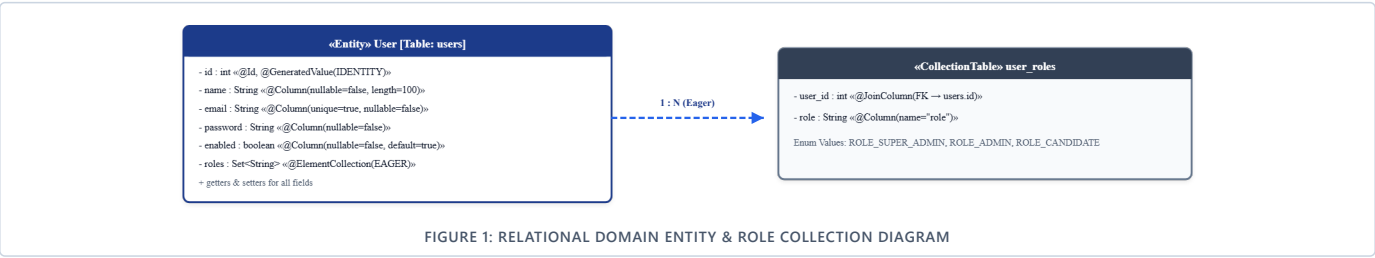
Complete File & Component Inventory (35 Source Files)

Layer / Package	Classes / Interfaces	Core Responsibility
Controllers	<code>AuthController</code> <code>AdminManagementController</code> <code>CandidateManagementController</code>	REST API endpoints for authentication, Super Admin CRUD, and Candidate administration.
Services	<code>UserService</code> <code>OtpService</code> <code>CustomUserDetailsService</code> <code>EmailService</code>	Business logic, Redis OTP caching, UserDetails synthesis, and async email dispatch.
Repositories & Entities	<code>UserRepository</code> <code>User</code>	JPA persistence layer mapping <code>users</code> and <code>user_roles</code> tables.
DTOs (Requests)	<code>LoginRequest</code> , <code>SignupRequest</code> , <code>OtpVerificationRequest</code> , <code>ResendOtpRequest</code> , <code>ForgotPasswordRequest</code> , <code>ResetPasswordRequest</code> , <code>ResetPasswordVerificationRequest</code> , <code>PendingSignup</code> , <code>PendingPasswordReset</code> , <code>AdminCreateRequest</code> , <code>CandidateUpdateRequest</code>	Validated input payloads with Bean Validation annotations (<code>@NotBlank</code> , <code>@Email</code> , <code>@Size</code> , <code>@Pattern</code>).
DTOs (Responses)	<code>JwtAuthenticationResponse</code> , <code>CurrentUserResponse</code> , <code>AdminResponse</code> , <code>CandidateResponse</code>	Structured API responses encapsulating tokens and user representations.
Security & Common	<code>SecurityConfig</code> , <code>JwtTokenProvider</code> , <code>JwtAuthenticationFilter</code> , <code>HtmlSanitizer</code> , <code>GlobalExceptionHandler</code> , <code>ErrorResponse</code> , Custom Exceptions (3)	JWT creation/validation, filter chain, XSS sanitization, and global REST exception advice.

2 UML Class Diagrams & Architecture Models

The diagrams below illustrate the exact class structures, field definitions, dependency injection graphs, and controller-to-service execution flows extracted directly from the compiled codebase.

A. Entity & Domain Model Diagram



B. Service Layer Dependency & Repository Mapping



3 Controller-to-Service Mapping & Auth API

The REST Controllers map client HTTP requests directly to service and security components. Below is the controller execution graph followed by the exhaustive API endpoint specification for `AuthController`.



FIGURE 3: CONTROLLER-TO-SERVICE INVOCATIONS

Authentication Endpoints (`AuthController` @ `/api/auth`)

Method & Path	Purpose	Auth / Role	Request Body / Params	Response Shape	Validation & Business Rules
POST <code>/login</code>	Authenticate user with credentials & issue JWT	Public	<code>LoginRequest</code> : <code>email</code> : String (req, @Email) <code>password</code> : String (req, ≥6)	200 OK : <code>JwtAuthenticationResponse</code> { token, tokenType }	Authenticates via <code>AuthenticationManager</code> ; validates active user state; issues 7-day HMAC-SHA256 JWT.
POST <code>/request-otp</code>	Initiate self-registration and send email OTP	Public	<code>SignupRequest</code> : <code>name</code> : String (req, 2–100, no HTML) <code>email</code> : String (req, @Email) <code>password</code> : String (req, ≥6)	200 OK : "Verification code sent." 409 CONFLICT if email exists	Generates 6-digit cryptographic OTP; hashes password; saves <code>PendingSignup</code> in Redis with 5-min TTL; dispatches email.
POST <code>/verify-otp</code>	Verify OTP, persist user, and auto-login	Public	<code>OtpVerificationRequest</code> : <code>email</code> : String (req, @Email) <code>code</code> : String (req, length=6)	200 OK : <code>JwtAuthenticationResponse</code> 401 UNAUTHORIZED if expired/invalid	Retrieves <code>PendingSignup</code> from Redis; verifies OTP; calls <code>UserService.createUser</code> (assigns <code>ROLE_CANDIDATE</code>); sends welcome email; purges Redis.
POST <code>/resend-otp</code>	Regenerate and resend signup OTP	Public	<code>ResendOtpRequest</code> : <code>email</code> : String (req, @Email)	200 OK : "OTP resent successfully." 404 NOT_FOUND if expired	Regenerates 6-digit OTP in existing Redis signup record; resets 5-minute TTL; sends updated email.
GET <code>/me</code>	Get authenticated user profile and roles	Bearer Auth	None (Reads <code>Authentication</code> from <code>SecurityContext</code>)	200 OK : <code>CurrentUserResponse</code> : { id, name, email, roles }	Extracts email from JWT subject; queries <code>UserRepository</code> ; returns active role permissions.
POST <code>/forgot-password</code>	Initiate password reset and dispatch reset OTP	Public	<code>ForgotPasswordRequest</code> : <code>email</code> : String (req, @Email)	200 OK : "Verification code sent." 404 NOT_FOUND if email missing	Verifies email existence; stores <code>PendingPasswordReset</code> in Redis with 5-min TTL; sends password reset email.
POST <code>/verify-reset-code</code>	Validate password reset OTP code	Public	<code>ResetPasswordVerificationRequest</code> : <code>email</code> : String (req, @Email) <code>code</code> : String (req, length=6)	200 OK : "Verification successful." 401 UNAUTHORIZED if invalid	Checks code against active Redis reset record without consuming the record yet.
POST <code>/resend-reset-code</code>	Resend password reset OTP code	Public	<code>ForgotPasswordRequest</code> : <code>email</code> : String (req, @Email)	200 OK : "Verification code resent." 404 NOT_FOUND if expired	Generates fresh OTP; updates Redis reset session; resets 5-min TTL; resends email.
POST <code>/reset-password</code>	Finalize password update with new credentials	Public	<code>ResetPasswordRequest</code> : <code>email</code> : String (req, @Email) <code>password</code> : String (req, ≥6)	200 OK : "Password reset successfully." 401 UNAUTHORIZED if expired	Verifies active reset session in Redis; encodes new password via BCrypt; updates User in DB; purges Redis reset key.

4 Admin & Candidate Management API Reference

Administrative operations are partitioned into two strict role-governed controllers to enforce separation of duties.

A. Admin Management Endpoints (`AdminController` @ `/api/admin/management`)

Access Control: Enforces class-level `@PreAuthorize("hasAuthority('ROLE_SUPER_ADMIN')")`. Standard administrators cannot manage or elevate other admin accounts.

Method & Path	Purpose	Auth / Role	Request Body / Params	Response Shape	Validation & Business Rules
GET <code>/</code>	List all system administrators	<code>ROLE_SUPER_ADMIN</code>	None	<code>200 OK</code> <code>List<AdminResponse></code> <code>[{ id, name, email, roles, enabled }]</code>	Filters <code>users</code> table for accounts possessing <code>ROLE_ADMIN</code> or <code>ROLE_SUPER_ADMIN</code> .
POST <code>/</code>	Provision a new administrator	<code>ROLE_SUPER_ADMIN</code>	<code>AdminCreateRequest</code> : <code>name</code> : String (req, 2–100) <code>email</code> : String (req, @Email) <code>password</code> : String (req, ≥6) <code>roles</code> : Set<String> (@NotEmpty)	<code>201 CREATED</code> <code>AdminResponse</code> <code>400 BAD_REQUEST</code> on duplicate email	Sanitizes name via <code>HtmlSanitizer</code> ; encodes password with <code>BCrypt</code> ; sets <code>enabled=true</code> ; saves with assigned roles.
PUT <code>//{id}</code>	Update administrator profile & roles	<code>ROLE_SUPER_ADMIN</code>	<code>id</code> : int (PathVariable) <code>AdminCreateRequest</code> (RequestBody)	<code>200 OK</code> <code>AdminResponse</code> <code>400 BAD_REQUEST</code> on failure	Updates name, roles; updates password if non-empty; saves to database.
PATCH <code>//{id}/reactivate</code>	Reactivate a deactivated admin	<code>ROLE_SUPER_ADMIN</code>	<code>id</code> : Long (PathVariable)	<code>204 NO_CONTENT</code>	Sets <code>enabled = true</code> on target user account.
DELETE <code>//{id}</code>	Soft-deactivate an admin account	<code>ROLE_SUPER_ADMIN</code>	<code>id</code> : Long (PathVariable)	<code>204 NO_CONTENT</code>	Sets <code>enabled = false</code> . Prevents account login while preserving historical audit logs.

B. Candidate Management Endpoints (`CandidateManagementController` @ `/api/candidate/management`)

Access Control: Enforces class-level `@PreAuthorize("hasAnyAuthority('ROLE_SUPER_ADMIN', 'ROLE_ADMIN')")`.

Method & Path	Purpose	Auth / Role	Request Body / Params	Response Shape	Validation & Business Rules
GET <code>/</code>	List all candidate test-takers	<code>ROLE_ADMIN</code>	None	<code>200 OK</code> <code>List<CandidateResponse></code> <code>[{ id, name, email, enabled }]</code>	Filters <code>users</code> table for accounts possessing <code>ROLE_CANDIDATE</code> .
PUT <code>//{id}</code>	Update candidate name/email/password	<code>ROLE_ADMIN</code>	<code>id</code> : Long (PathVariable) <code>CandidateUpdateRequest</code> (RequestBody)	<code>200 OK</code> <code>CandidateResponse</code>	Sanitizes name; updates email; updates password if non-blank; saves changes.
PATCH <code>//{id}/reactivate</code>	Reactivate candidate assessment access	<code>ROLE_ADMIN</code>	<code>id</code> : Long (PathVariable)	<code>204 NO_CONTENT</code>	Sets <code>enabled = true</code> . Re-enables assessment participation and portal login.
DELETE <code>//{id}</code>	Soft-deactivate a candidate account	<code>ROLE_ADMIN</code>	<code>id</code> : Long (PathVariable)	<code>204 NO_CONTENT</code>	Sets <code>enabled = false</code> . Locks candidate access without deleting prior assessment records.

C. Assessment Session & Test-Taking API (AdminAssessmentController & CandidateAssessmentController)

Method & Path	Handler & Controller	Authorization	Request Payload / Params	Response Type	Security & Error Rules
POST /api/admin/attempts	createAttempt() AdminAssessmentController	ROLE_ADMIN	AdminAttemptCreateRequest (candidateId)	200 OK AssessmentAttempt	400 if disabled; 409 if non-SCORED attempt exists. Creates 4 locked sessions.
GET /api/admin/attempts	getCandidateAttempts() AdminAssessmentController	ROLE_ADMIN	Query: candidateId=Long	200 OK List<AssessmentAttempt>	Admin audit view of candidate progress across all batteries.
GET /api/attempts/me/pending	getPendingAttempt() CandidateAssessmentController	ROLE_CANDIDATE	None (JWT Principal)	200 OK AssessmentAttempt	Resolves active non-SCORED attempt; 404 if none assigned.
GET /api/attempts/{token}	getAttempt() CandidateAssessmentController	ROLE_CANDIDATE	Path: token	200 OK AssessmentAttempt	403 Forbidden if attempt.candidate.id != authenticatedUser.id .
POST /api/attempts/{token}/start	startAttempt() CandidateAssessmentController	ROLE_CANDIDATE	Path: token	200 OK AssessmentAttempt	409 if already started; starts battery 0 (PQ10) and initiates Redis timer.
POST /api/attempts/battery-sessions/{id}/heartbeat	heartbeat() CandidateAssessmentController	ROLE_CANDIDATE	HeartbeatRequest (responses list)	200 OK { remainingTimeSeconds }	403 if battery not matching current sequence; buffers to Redis; returns remaining time.
POST /api/attempts/battery-sessions/{id}/submit	submitSession() CandidateAssessmentController	ROLE_CANDIDATE	Path: id	200 OK AssessmentAttempt	Flushes buffer to DB, sets SUBMITTED, auto-advances currentBatteryIndex.
GET /api/attempts/{token}/report	getReport() CandidateAssessmentController	ROLE_CANDIDATE	Path: token	200 OK DossierReport	404 if state != SCORED. Combined 5-page dossier for completed attempt.

5 Architectural Observations & Quality Audit

A non-intrusive static code analysis of the existing codebase identified the following architectural, type consistency, and error-handling characteristics. As per the reference constraints, these are documented below for review:

- 1. Primary Key Type Inconsistency (int vs. Long)**

In `User.java` , the primary key `id` is declared as primitive `int` . However, `UserRepository` extends `JpaRepository<User, Long>` , and several service methods (e.g. `deactivateUser(Long id)`) and controller path variables use `Long` while others use `int` (e.g. `AdminManagementController.updateAdmin(@PathVariable int id)`).

Impact: Functionally works due to autoboxing and `Long.valueOf(id)` casting, but standardizing on `Long` across entities, DTOs, and controllers is recommended during domain expansion.
- 2. Exception Handling Consistency Across Controllers**

In `AdminManagementController` , `addAdmin` and `updateAdmin` methods wrap execution in manual `try-catch` blocks returning `ResponseEntity.status(HttpStatus.BAD_REQUEST).body(e.getMessage())` . Conversely, `CandidateManagementController` and other endpoints allow exceptions to bubble up to `GlobalExceptionHandler` .

Impact: Manual try-catch blocks produce plaintext error strings rather than the standardized JSON structure defined in `ErrorResponse.java` .
- 3. Generic RuntimeException Usage in UserService**

In `UserService.java` , entity lookup failures throw generic `new RuntimeException("User not found")` or `new RuntimeException("Admin account not found")` instead of the custom `ResourceNotFoundException` .

Impact: `GlobalExceptionHandler` catches these as general `500 INTERNAL_SERVER_ERROR` instead of returning a clean `404 NOT_FOUND` status.
- 4. CORS Declaration Granularity**

`AuthController` contains a class-level `@CrossOrigin(origins = "**")` annotation, whereas `AdminManagementController` and `CandidateManagementController` rely on the centralized `CorsConfigurationSource` bean in `SecurityConfig.java` .

Impact: Centralized CORS in `SecurityConfig` is the authoritative policy; the wildcard annotation on `AuthController` is redundant.

Summary for Next Phase (Domain Implementation)

The authentication and user management foundation is robust, secure, and compiling with 100% test success. These minor architectural observations can be naturally aligned when constructing the subsequent Psychometric Item Bank, Testing Session Engine, and Scoring Pipeline.